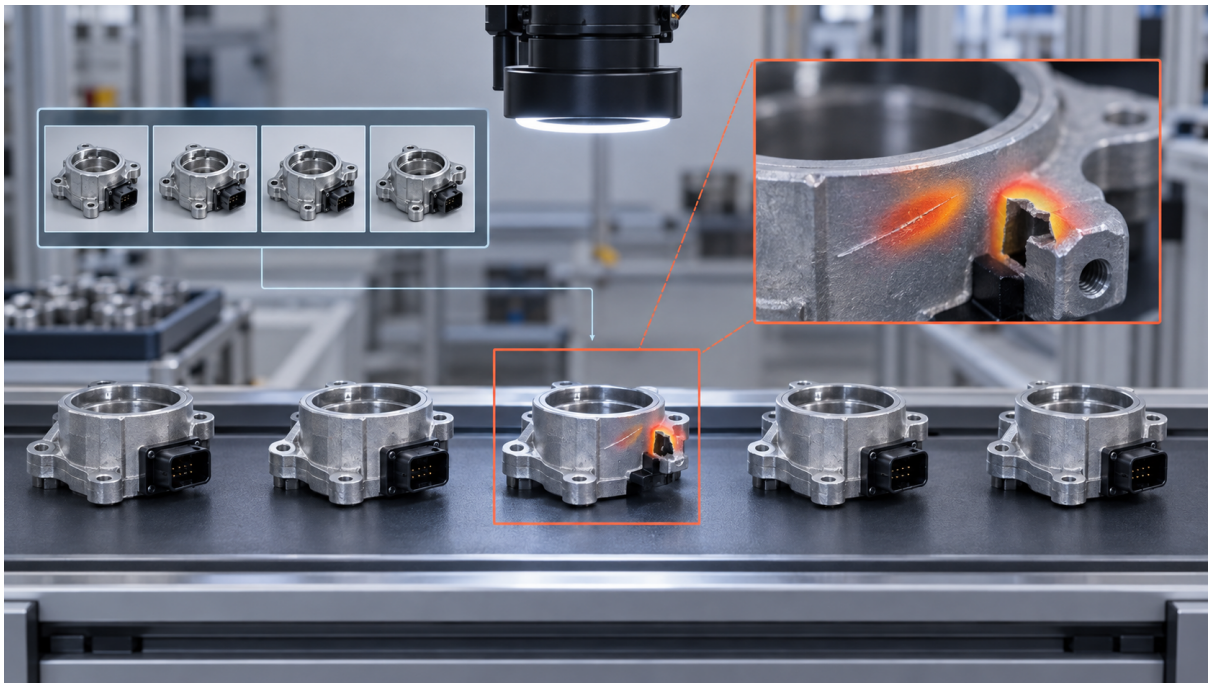
Project 2: Few-Shot Industrial Visual Anomaly Detection

AIAA 3201—Introduction to Computer Vision

Term Project Instruction

Spring 2027



1 Overview

Core Task: Given only a small set of defect-free reference images, students are required to design a system that identifies whether a test image is anomalous and localizes the anomalous region at pixel level. The project studies the evolution from hand-crafted inspection and reconstruction-based models to training-free foundation-model features and vision-language prompting. The system must also remain reliable when illumination, viewpoint, background, or product category changes.

Key Concepts:

- **Unsupervised Anomaly Detection:** Learning normality without labeled defects.
- **Foundation Model Features:** Patch-level representation and nearest-neighbor retrieval.
- **Vision-Language Models:** Normal/abnormal prompt learning and zero-shot transfer.
- **Reliability:** Domain shift, calibration, uncertainty, and false-alarm control.

Group Size: 2 students per group.

2 Submission Requirements

- **PDF Report (Mandatory):**
  - **Format:** Use the CVPR LaTeX Template. The length must be **6–8 pages** (excluding references).
  - **ArXiv Upload:** You are required to upload your final report to **arXiv**. Remember to change the "REVIEW version" to "CAMERA-READY version" in the template. Please clearly indicate your arXiv ID on the first page of the Canvas submission.
  - **Content Requirements:**
    1. **Abstract & Introduction:** Background, motivation, problem definition, and solution overview. **Explicitly include your public GitHub repository link at the end of the abstract.**
    2. **Related Work:** Review and cite at least all papers listed in the Recommended Reading List (Section 4). Expanding beyond this list is encouraged.
    3. **Method:** Explain the technical roadmap **in your own words**. Use high-quality pipeline diagrams and visual examples to support the explanation.
    4. **Experiments:** Present quantitative results in tables, qualitative comparisons in figures, ablation studies, and categorized failure cases.
    5. **Conclusion:** Summarize the findings, limitations, and specific directions for future work.
- **Code (Mandatory):** Upload the code to a public **GitHub repository** with a clear `README.md`. Include dependencies, data preparation, model weights, training and evaluation commands, and representative visual results. Do not submit raw code files to Canvas.
- **Interactive Demo (Mandatory):** Submit a runnable demo or a recorded demonstration that accepts a product image and displays an image-level anomaly score, an OK/NG decision, and a pixel-level heatmap. Pack the demo video and representative outputs into `anomaly_demo.zip`.

Important Tips for Success

- **Method Flexibility:** You may follow the suggested roadmap or design another technically sound method. Strict adherence to the Implementation Roadmap is not required if the core task and evaluation protocol are fulfilled.
- **Completeness:** All three parts are mandatory. If Part 3 does not improve the main metric, analyze why and provide controlled ablations and failure cases. Ignoring Part 3 will result in an "Incomplete" grade.
- **Visuals:** Clean pipeline diagrams, prediction overlays, zoomed comparisons, and failure visualizations are essential in a Computer Vision course and will substantially affect the presentation score.
- **Dataset Scope:** Completing the specified VisA benchmark ensures a passing grade. High-scoring projects should evaluate all 12 VisA categories and study robustness across its object domains, rather than downloading an additional dataset.
- **Fair Comparison:** Use identical data splits, preprocessing, input resolution, and evaluation code when comparing methods. Report runtime, peak GPU memory, and all test-time refinements.
- **Citations:** Cite all datasets, pretrained models, external repositories, and borrowed implementation components correctly.

3 Guide to Start

To help you get started efficiently, we recommend the following workflow:

1. **Step 1: Literature Review.** Read the papers in the Recommended Reading List (Section 4). Understanding normal-only learning, patch embeddings, nearest-neighbor anomaly scoring, and image-level versus pixel-level metrics is essential before coding.
2. **Step 2: Code Familiarization.** Clone and run the official repositories for AnomalyDINO and AnomalyCLIP. Run their provided demos first, then verify the input/output format and pretrained checkpoint on one sample.
3. **Step 3: Experimentation.** Download the datasets from the official links in Section 6.1. Run the baselines and official pretrained methods using the stated category subset, data split, and random seeds. Cache frozen backbone features and save anomaly maps and error cases for every method.
4. **Step 4: Writing & Submission.** Organize quantitative results, ablations, and failure cases into tables and figures. Write the report with the CVPR template, cite all references, upload the camera-ready manuscript to arXiv, and submit the required files to Canvas.

4 Recommended Reading List

- **Foundations:** MVTec AD [1], PaDiM [2], PatchCore [3]
- **Reconstruction & Synthesis:** DRAEM [4], CutPaste [5]
- **Foundation Models:** DINOv2 [6], AnomalyCLIP [7], AnomalyDINO [8]
- **Efficiency & Robustness:** EfficientAD [9], MVTec AD 2 [10]

5 Implementation Roadmap

5.1 Part 1: The Baseline – "Classical Normality Modeling"

Goal: Build a complete normal-only inspection pipeline and understand how alignment, representation, and threshold selection affect anomaly detection.

1. Image Alignment & Hand-Crafted Difference

- **Step 1: Registration.** Align each test image to the closest normal template using ECC, feature-based homography, or another justified method.
- **Step 2: Difference Map.** Compute absolute RGB/gray differences, SSIM differences, gradients, or local texture descriptors. Use smoothing and morphology to obtain anomaly regions.
- **Step 3: Thresholding.** Choose thresholds only on normal training/validation images. Explain how the desired false-positive rate determines the operating point.

2. Reconstruction Baseline: Convolutional Autoencoder

- **Algorithm:** Train a compact autoencoder using only normal images. Use pixel, SSIM, or perceptual reconstruction error as the anomaly map.
- **Key Analysis:** Compare template difference and learned reconstruction on texture defects, structural defects, and lighting changes.

Expected Result: The baseline should detect obvious scratches and missing components but will often confuse pose, background, and illumination changes with defects.

5.2 Part 2: SOTA Reproduction – "Foundation Features & Prompt Learning"

Goal: Reproduce a recent, openly released method that works with very few normal reference images and produces both image-level and pixel-level predictions.

1. Few-Shot Anomaly Detection: AnomalyDINO [8]

- **System:** Extract DINOv2 patch features from 1/2/4/8 normal references. Construct a memory bank and score each test patch by its nearest normal neighbor.
- **Required Reproduction:** Use the official code and pretrained DINOv2 backbone. Report the 1/2/4/8-shot curves on the mandatory VisA categories with at least three random seeds.
- **Key Learning Point:** Analyze why frozen semantic features can localize unseen defects without task-specific training, and where coarse patch resolution fails.

2. Zero-Shot Alternative: AnomalyCLIP [7]

- **System:** Use object-agnostic normal and abnormal text prompts to produce image scores and anomaly maps without target-category training.
- **Comparison:** Compare vision-only few-shot retrieval with text-guided zero-shot detection under the same image preprocessing and metrics.

Expected Result: AnomalyDINO should provide a strong training-free few-shot baseline; AnomalyCLIP may generalize across categories but can be sensitive to prompts and fine-grained defects.

5.3 Part 3: Exploration – "Robustness & Reliable Decisions"

Goal: Identify a concrete failure mode and implement at least one justified improvement. The main backbone should remain frozen unless a small adapter is explicitly studied.

- **Direction A (Multi-Scale Features):** Fuse patch features from multiple DINOv2 layers or image scales. Test whether fine features improve tiny scratches without increasing background false positives.
- **Direction B (Background Suppression):** Use foreground masks, PCA masks, or segmentation prompts to remove unstable background patches from the memory bank and anomaly map.
- **Direction C (Synthetic Defects):** Generate CutPaste or texture-based pseudo-defects and train only a lightweight scoring head. Study which synthetic anomalies transfer to real defects.
- **Direction D (Prompt & Feature Fusion):** Combine AnomalyCLIP prompt scores with AnomalyDINO visual distances using a calibrated gate or uncertainty-aware weighting.
- **Direction E (Memory & Efficiency):** Compress the normal memory bank with clustering, coreset selection, or product-specific prototypes. Report accuracy, runtime, and memory trade-offs.
- **Direction F (Selective Prediction):** Calibrate anomaly scores and allow an "uncertain" output. Evaluate risk-coverage curves and the false-negative rate at a fixed false-positive rate.

6 Dataset & Evaluation

6.1 Datasets and Required Downloads

1. **VisA (The Only Required Dataset):** Download the complete 1.80 GB public archive directly from AWS. No account, login, or request form is required. VisA contains 10,821 images from 12 categories with image- and pixel-level annotations under CC BY 4.0 [11]. For the common course benchmark, use `candle`, `capsules`, `cashew`, `chewinggum`, `fryum`, and `pipe_fryum`; groups may evaluate all 12 categories for a stronger result. Direct dataset archive | Official AWS dataset record

2. **Starter Code and Weights (Required):** Use the official repositories. AnomalyDINO documents the VisA folder structure and can run FAISS on CPU; AnomalyCLIP provides pretrained checkpoints and JSON-generation scripts. DINOv2 weights are obtained through the official code path. AnomalyDINO | AnomalyCLIP | DINOv2

6.2 Metrics (Mandatory)

- **Image-Level Detection:** AUROC and Average Precision (AP). Report the false-positive rate at the selected operating threshold.
- **Pixel-Level Localization:** Pixel AUROC, pixel AP, and AUPRO. Use the official masks and state the connected-region integration limit.
- **Few-Shot Stability:** Mean and standard deviation over at least three random reference selections for 1/2/4/8-shot settings.
- **Efficiency:** Feature extraction time, scoring time, peak GPU memory, and memory-bank size.
- **Qualitative Evaluation:** Show input, ground-truth mask, anomaly map, and thresholded prediction. Categorize at least 20 false positives/negatives by cause.

Note: Report image-level and pixel-level metrics separately. Selecting a threshold on the test labels is prohibited.

References

[1] Bergmann, P., et al. "MVTec AD—A Comprehensive Real-World Dataset for Unsupervised Anomaly Detection." CVPR 2019.

[2] Defard, T., et al. "PaDiM: A Patch Distribution Modeling Framework for Anomaly Detection and Localization." ICPR Workshops 2021.

[3] Roth, K., et al. "Towards Total Recall in Industrial Anomaly Detection." CVPR 2022.

[4] Zavrtanik, V., et al. "DRAEM—A Discriminatively Trained Reconstruction Embedding for Surface Anomaly Detection." ICCV 2021.

[5] Li, C.-L., et al. "CutPaste: Self-Supervised Learning for Anomaly Detection and Localization." CVPR 2021.

[6] Oquab, M., et al. "DINOv2: Learning Robust Visual Features without Supervision." TMLR 2024.

[7] Zhou, Q., et al. "AnomalyCLIP: Object-Agnostic Prompt Learning for Zero-Shot Anomaly Detection." ICLR 2024.

[8] Damm, S., et al. “AnomalyDINO: Boosting Patch-Based Few-Shot Anomaly Detection with DINOv2.” WACV 2025.

[9] Batzner, K., et al. “EfficientAD: Accurate Visual Anomaly Detection at Millisecond-Level Latencies.” WACV 2024.

[10] Heckler-Kram, L., et al. “The MVTec AD 2 Dataset: Advanced Scenarios for Unsupervised Anomaly Detection.” IJCV 2026.

[11] Zou, Y., et al. “SPot-the-Difference Self-Supervised Pre-Training for Anomaly Detection and Segmentation.” ECCV 2022.

[12] Kirillov, A., et al. “Segment Anything.” ICCV 2023.